

MINI ASSESSMENT

Name: V. Akash Varma

Reg. No: 99230041018

Dept: CSE

Q1. If an API returns 429, what should your program do?

A 429 status code means “**Too Many Requests**” — the client has exceeded the API's rate limit.

- Check for a **Retry-After** header and wait for that duration before retrying.
- Implement **exponential backoff** (wait progressively longer between retries) if no header is present.
- Limit the number of retry attempts to avoid an infinite loop.
- Log the event and, if retries are exhausted, fail gracefully and notify the user.

Q2. If `response.json()` fails, what should your program do?

`response.json()` fails (raises an exception, e.g. `JSONDecodeError`) when the response body is empty, malformed, or not valid JSON (often an HTML error page instead). The program should:

- Wrap the call in a **try/except** block to catch the decoding error.
- Check **`response.status_code`** and **`response.text`** first to understand what was actually returned.
- Avoid letting the exception crash the whole program — handle it and return a safe default or error message.
- Log the raw response for debugging purposes.

Q3. If a JSON key is missing, why might `.get()` be safer than `[]`?

Using square brackets, e.g. `data['key']`, raises a `KeyError` and crashes the program if the key does not exist. Using `data.get('key')` is safer because:

- It returns **`None`** (or a default value you specify) instead of raising an error.
- It allows the program to continue running smoothly even with incomplete or inconsistent JSON data.
- You can supply a fallback: `data.get('key', 'default_value')`.
- It leads to more robust, production-ready code that handles real-world API responses gracefully.